

search-architecture

Phase 0 Research — Semantic Index and Search Architecture

Feature: 001-workshop-curriculum-platform **Scope:** User Story 3; FR-014...FR-020, FR-026, FR-027, FR-037; SC-005, SC-006, SC-007, SC-008, SC-016. **Date measured:** 2026-08-31, 22:00–22:40 CEST. **Constraints taken as given (not relitigated):** D1 local/internal only · D2 corpus = workshop/ + the vasic monorepo · FR-037 persisted stable identifiers · the SC latency and quality targets.

Sibling research documents, deliberately out of scope here: research/transcription.md (how audio becomes text) and research/llm-bridging.md (how an answer model is attached). This document stops at the boundary of both: it specifies the *passage contract* that transcription must emit, and the *retrieval contract* that answering must consume.

0. Method, and what could not be measured

Every number below was produced by a command run against this machine on 2026-08-31. Commands and raw output are reproduced inline and in Appendix A.

Two conditions constrained the measurements and are stated up front because they change how the numbers should be read:

1. A lumen index -f <ext-volume-host>/Projects/vasic run started at 21:12 CEST was **still running throughout**. All embedding latencies below were therefore taken **under concurrent index load**. That is a realistic operating condition (it is exactly what happens after every chapter ingest), but it is not the idle case.

2. The live index was treated as **read-only**. Every read used `sqlite3 "file:...?mode=ro"`. All write experiments were done against a throwaway two-file project in the scratchpad.

UNVERIFIED items are collected in Appendix C. Nothing in this document reports a state that was not observed.

Finding 1 — What the existing Lumen capability actually provides

Decision

Lumen is adopted as the **embedding and vector-retrieval engine** for the corpus it already covers, and for nothing else. It is treated as a *component*, not as the search system. Specifically:

- **Use it for:** turning markdown and source code into 768-dimensional vectors and returning nearest-neighbour matches as `(file_path, start_line, end_line, symbol, kind, score)`.
- **Do not use it for:** passage identity, cross-references, autocomplete, lexical/keyword matching, an HTTP API, coverage of file types it does not parse, or the three-state availability contract.

Rationale — measured facts

Version and shape. `lumen version` → 0.0.41. The lumen on PATH is a bash launcher at `<home>/local/bin/lumen` that resolves the highest installed plugin version; the real binary is `<home>/claude-shared/plugins/cache/claude-plugins-official/lumen/0.0.41/bin/lumen-linux-amd64` (stripped Go ELF, module path `github.com/ory/lumen`).

CLI surface — the complete command set:

```
$ lumen --help
Available Commands:
  completion  Generate the autocompletion script for the specified shell
  hook        Hook handlers for AI coding agent integration
  index       Index a project for semantic search
  purge       Remove lumen index data
  search      Search an indexed project for semantically similar code
  stdio       Start the MCP server on stdin/stdout
  version     Print the lumen version
```

```

$ lumen index --help
Usage: lumen index <project-path> [flags]
  -b, --backend string    embedding backend ("ollama" or
"lmstudio")
  -f, --force              force full re-index
  -m, --model string      embedding model (default:
$LUMEN_EMBED_MODEL or ordis/jina-embeddings-v2-base-code)

$ lumen search --help
Usage: lumen search <query> [flags]
  -b, --backend string    embedding backend
  --cwd string            project root when path is a subdirectory
  -f, --force              force full re-index before searching
  --max-lines int         truncate snippets at N lines (0 =
unlimited)
  --min-score float       minimum score threshold (-1 to 1)
  -m, --model string      embedding model override
  -n, --n-results int     max results to return (default 8)
  -p, --path string       directory to search (default: cwd)
  --summary                omit code snippets, return location only
  --trace                 print per-phase timing to stderr

```

There is **no** subcommand to list, dump, or address a chunk by identifier; no JSON output flag; no incremental “index only this file” command; no lexical/keyword mode.

MCP surface — exactly three tools, confirmed by driving `lumen stdio` directly over JSON-RPC:

```

$ python3 mcpprobe.py
initialize 20 ms
tools: ['health_check', 'index_status', 'semantic_search']

```

`semantic_search(query, path?, cwd?, limit?, min_score?, max_lines?, summary?), index_status(path), health_check()`. No tool exposes chunk identifiers, cross-references, or suggestions.

Index schema (live index for this repo, `~/.local/share/lumen/21bf1507a8925bcf/index.db`, 338 MB):

```

CREATE TABLE files (path TEXT PRIMARY KEY, hash TEXT NOT NULL);
CREATE TABLE project_meta (key TEXT PRIMARY KEY, value TEXT NOT
NULL);
CREATE TABLE chunks (
  id          TEXT PRIMARY KEY,
  file_path   TEXT NOT NULL REFERENCES files(path),
  symbol      TEXT NOT NULL,
  kind        TEXT NOT NULL,
  start_line  INTEGER NOT NULL,
  end_line    INTEGER NOT NULL

```

```
);
CREATE INDEX idx_chunks_file_path ON chunks(file_path);
CREATE VIRTUAL TABLE vec_chunks USING vec0 (
    id TEXT PRIMARY KEY,
    embedding float[768] distance_metric=cosine
);
```

files.hash is the **sha256 of the file's bytes** — verified: files.hash for workshop/README.md is 3f83a338...feca, identical to sha256sum of that file. chunks.id is a 16-hex-character (64-bit) value; all 58,734 rows are exactly 16 characters and 6.13 % begin with 0 (expected 6.25 % for a zero-padded hex hash), so it is a padded 64-bit digest, not a counter. The vectors live in a separate sqlite-vec vec0 virtual table joined to chunks **by that same id** — meaning the id is the only key linking a chunk's metadata to its vector.

Project metadata for this repo:

```
vec_dimensions=768
embedding_model=ordis/jina-embeddings-v2-base-code
project_path=<ext-volume-host>/Projects/vasic
last_indexed_at=2026-08-31T17:51:01Z
total_files=2510
root_hash=11e7034b26c9b83c...
```

Embedding model and dimension: ordis/jina-embeddings-v2-base-code, 768 dims, cosine distance, served by ollama at <http://localhost:11434>, **100 % CPU** (ollama ps shows PROCESSOR 100% CPU, CONTEXT 8192). CPU-only is deliberate: scripts/ollama-vulkan-remediation.sh forced it after a Vulkan/iGPU fault silently wrote 758 duplicate vectors into this index on 2026-08-26.

Chunking. Symbols in the binary show a MultiChunker dispatching to MarkdownChunker, GoAST, TreeSitterChunker, StructuredChunker (YAML/JSON key paths) and SvelteChunker, followed by mergeUndersizedChunks / splitOversizedChunks / createSubChunks governed by a max_chunk_tokens setting. Empirically, over the 58,734 live chunks:

kind	count
section (markdown heading block)	54,445
function	2,919
method	447
var	347
type	297

kind	count
const	172
document (whole structured file)	84
interface	23

Span statistics: **min 1 line, mean 5.8 lines, max 133 lines**; 41,300 chunks (70 %) are ≤ 5 lines. Markdown is cut at headings — one chunk per heading, from the heading line to the line before the next heading.

Can it index non-code content? — Yes for markdown, no for most other things. This is measured, not inferred.

Markdown is not merely supported, it *dominates* this index: 1,810 of 2,510 indexed files are .md, and 92.7 % of all chunks are markdown sections. A transcript published as markdown will be indexed with no special handling.

But the file-type coverage is a hard allowlist, and the exclusions matter:

```
$ sqlite3 "file:$D?mode=ro" "select count(*) from files where path
like '%.sh';" → 0
... same query for .html → 0    .css → 0    .txt → 0    .svg →
0    .mmd → 0
    .puml → 0    .vtt → 0    .srt → 0    .pdf → 0    .mp4 → 0    .rb →
0    .toml → 0
$ git ls-files '*.sh' | wc -l → 55
$ git ls-files '*.html' | wc -l → 146
$ git ls-files '*.css' | wc -l → 32
```

Those 55 shell scripts, 146 HTML files and 32 stylesheets are tracked in the repository's root module and **none of them are in the index**. This is exclusion, not absence. The extensions actually present are md go json mjs py js yaml ts yml; the binary additionally carries .tsx .jsx .cpp .cxx .hpp .php .mdx string constants and tree-sitter queries for C/C++/C#/Rust.

Consequence for FR-016: “index all curriculum content types ... source code” cannot be satisfied by Lumen alone, because this project's operational surface is bash. That gap is closed in Finding 4, not wished away.

Can it be queried programmatically with structured results? — Yes, but the structure is weak.

Both the CLI and the MCP tool return the *same* semi-structured text blob. Real output:

```
$ lumen search "splitting a video into pieces" -p <proj> --trace  
-n 3
```

Found 1 results:

```
<result:file filename="a.md">  
  <result:chunk line-start="5" line-end="8" symbol="Alpha"  
kind="section" score="0.69">  
# Alpha
```

The recording pipeline splits the video into parts.

```
  </result:chunk>  
</result:file>
```

It is XML-shaped but is not declared as an API, has no schema, is not valid XML in general (chunk bodies are raw source, unescaped), and — **critically** — **it does not contain chunks.id**. The only join key a consumer gets is (file_path, line-start, line-end, symbol, kind), which is a *positional* locator. Any consumer that wants stable identity must supply it from elsewhere.

Alternatives considered

- **Replace Lumen with a project-owned sqlite-vec store.** Rejected. The spec's Assumptions section commits to reusing the existing capability ("builds on it rather than introducing a competing one"), Lumen already carries 58,734 embedded chunks for this corpus at a cost of roughly 15 hours of CPU embedding, and it already solves chunking for eight languages plus markdown. Rebuilding that is weeks of work for no retrieval-quality gain.
 - **Fork/patch Lumen to add file types and stable ids.** Rejected. It ships as a stripped vendor binary inside a Claude Code plugin cache that is replaced on every plugin upgrade (~/.local/bin/lumen exists precisely because version-pinned paths break). A fork would be silently overwritten and would fail the repository's own portability audits.
 - **Use Lumen for everything including identity.** Rejected on the evidence in Finding 2.
-

Finding 2 — Identifier stability: Lumen chunk ids are content-derived and **MUST NOT** be used as passage identifiers

This is the single most consequential finding in this document. SC-016 requires 100 % of cross-references and citations to survive a transcript correction plus re-index. Built on Lumen's ids, that figure would be **0 % for every corrected passage**, and the failure would be silent.

Decision

Build an identifier layer. The system mints its own **Passage Identifier (PID)** at ingest, persists it in the repository, and never derives it from position or content. Lumen's `chunks.id` is treated as an internal implementation detail of a disposable index and is never stored, cited, or linked to. (It is not even reachable — see Finding 1: neither the CLI nor the MCP tool returns it.)

Rationale — the experiment and its result

A controlled three-run experiment was executed against a throwaway git project in the scratchpad, using the same binary, model and backend as the live index.

Fixture `a.md` (three heading sections; `b.md` is an unchanging control):

```
1  # Alpha
2
3  The recording pipeline splits the video into parts.
4
5  # Beta
6
7  A passage that will recieve a correction later on.      ←
deliberate typo
8
9  # Gamma
10
11 The third section describes cross references between passages.
```

Run 1 — baseline index.

Done. Indexed 2 files, 4 chunks in 28.583s.

```
d1b10e7c0bb67aa6 | a.md | Alpha | 1-4
4b98e295c112a429 | a.md | Beta  | 5-8
```

0c15aed47e3f1fab | a.md | Gamma | 9-12
2ddebd7cddded5d | b.md | Control | 1-4

Run 2 — a one-word correction inside Beta (recieve → receive). Line numbers unchanged.

Reason: root hash changed
Files: 0 added, 1 modified, 0 removed (2 total in project)
Done. Indexed 1 files, 3 chunks in 29.927s.

d1b10e7c0bb67aa6 | a.md | Alpha | 1-4 ← unchanged
610e608a50f273a6 | a.md | Beta | 5-8 ← CHANGED (was
4b98e295c112a429)
0c15aed47e3f1fab | a.md | Gamma | 9-12 ← unchanged
2ddebd7cddded5d | b.md | Control | 1-4 ← unchanged

Run 3 — a new section prepended. Every later section shifts down four lines; their text is untouched.

Files: 0 added, 1 modified, 0 removed (2 total in project)
Done. Indexed 1 files, 4 chunks in 48.075s.

1979ad59adf7bfe5 | a.md | Zero | 1-4 ← new
d1b10e7c0bb67aa6 | a.md | Alpha | 5-8 ← id unchanged,
position moved
610e608a50f273a6 | a.md | Beta | 9-12 ← id unchanged,
position moved
0c15aed47e3f1fab | a.md | Gamma | 13-16 ← id unchanged,
position moved
2ddebd7cddded5d | b.md | Control | 1-4 ← unchanged

Conclusion, stated as fact:

Property	Result
Stable across position shift (re-cutting, inserting earlier content)	YES — ids survived a four-line shift of every chunk
Stable across content correction (fixing a typo)	NO — a single-character edit re-issued the identifier
Therefore the id is	content-derived, position-independent

FR-037 forbids identifiers that are “neither positional nor content-derived”. Lumen’s id is *exactly* one of the two forbidden kinds. It is the kind that breaks on the operation this feature performs most often: FR-038 explicitly provides for a human correction layer over machine transcript output, and every correction would re-issue the identifier of

the corrected passage while leaving every neighbouring passage untouched — so the damage is scattered, partial, and invisible in aggregate statistics.

Two independent reasons make this decisive rather than merely inconvenient:

1. **The id is not exposed anyway.** Neither `lumen_search` nor `semantic_search` returns it (Finding 1). A citation built on Lumen output would have to be keyed on `(path, line-start, line-end)` — a purely positional key, which run 3 shows moves whenever anything above it changes. Both available keys fail FR-037, in opposite directions.
2. **The reverse-engineering attempt failed, and that is itself a reason.** 120 candidate derivations (sha256/sha1/md5/blake2b/blake2s/fnv-1a truncations over 20 field combinations of path, symbol, kind, line range and body text) were tested against four known id/content pairs; none matched. The formula is undocumented, unrecovered, and belongs to a vendor binary that is replaced on plugin upgrade. Depending on it would be depending on an unversioned private detail.

The identifier layer

PID format. A ULID minted at ingest, namespaced:

`wp_01JQ8Z7K3M4N5P6Q7R8S9T0V1W`. ULIDs are 26 Crockford-base32 characters, lexicographically sortable by mint time, need no central coordinator, and carry no positional or content information. (UUIDv7 is an equivalent choice; ULID is preferred only because the text form is URL- and citation-safe without hyphens.)

Where the PID lives — three places, one authority:

1. **In the source artifact itself**, for every passage in a document the system authors (transcripts, chapter pages, diagram descriptions). A markdown-invisible anchor immediately above the passage:

```
<!-- pid: wp_01JQ8Z7K3M4N5P6Q7R8S9T0V1W -->
### 00:14:32 – Milos: why the archive is split into parts
```

Text of the passage ...

This is the load-bearing design decision. The identifier is **authored once, by the machine, at ingest, into the artifact** — so it is neither positional nor content-derived, it is committed to git alongside the text it names, and it survives *any* subsequent

operation including a complete rebuild of every index from scratch. Correcting the passage edits the text under the anchor; the anchor does not move. Re-cutting the transcript moves the anchor with its passage.

2. **In the passage registry** — workshop/curriculum/passages.db (SQLite, committed), the authoritative record:

```
CREATE TABLE passage (
  pid          TEXT PRIMARY KEY,
    -- ULID, minted once, never recomputed
  chapter      TEXT NOT NULL,
  source_path  TEXT NOT NULL,          -- repo-relative
  source_kind  TEXT NOT NULL,          -- transcript|doc|
    code|diagram|pdf|media
  locator      TEXT NOT NULL,          -- JSON:
    {line_start,line_end} | {t_start,t_end} | {symbol}
  layer        TEXT NOT NULL,
    -- machine|human      (FR-038)
  confidence   REAL,
    -- ASR confidence      (FR-003)
  provenance   TEXT NOT NULL,          -- asr|authored|
    extracted|ocr
  content_hash TEXT NOT NULL,          -- CHANGE DETECTION
    ONLY – never identity
  minted_at    TEXT NOT NULL,
  superseded_by TEXT REFERENCES passage(pid),
  redacted_at  TEXT                    -- FR-039
);
CREATE TABLE passage_alias (old_pid TEXT PRIMARY KEY, pid TEXT
  NOT NULL REFERENCES passage(pid));
```

content_hash exists and is named so that no future reader mistakes it for identity: it answers “did this change?”, never “which passage is this?”.

3. **In a plain-text sidecar** — workshop/curriculum/passages.jsonl, one record per line, committed. The SQLite file is a build artifact of the JSONL, not the other way round. Rationale: a binary database is unreviewable in a diff and unmergeable in a conflict; the JSONL makes every PID mint and every provenance change visible in git diff, which is what makes SC-016 auditable by a human rather than only by a script.

Reconciliation at re-ingest — an ordered ladder, first match wins:

1. The source artifact carries a pid: anchor → reuse that PID. (Covers all authored/transcribed content. This is the common case and it is exact.)

2. The source is code and the (source_path, symbol_path, kind) triple matches an existing passage → reuse. Symbol path (internal/store/store.go#Store.Search) survives edits to a function body; it does not survive a rename.
3. A rename/move is detected (same content_hash, different path or symbol) → reuse the PID and record the old key in passage_alias.
4. Otherwise → mint a new PID, and if the artifact is one the system authors, **write the anchor back into it** in the same commit.

Why code passages are honestly weaker. Rule 2 keys on symbol path, which is positional in the namespace sense. Renaming a Go function re-keys its passage; rule 3 catches it only when the body is byte-identical. Littering source code with PID comments to fix this would be a worse cure than the disease. The honest statement, which belongs in the FR-031 documentation: **transcript, documentation and diagram passages have exact, permanent identity; code passages have best-effort identity with an alias table, and a rename plus an edit in the same commit can orphan a code cross-reference.** Orphans resolve to a tombstone that says so, never to a wrong passage.

Citation and cross-reference resolution always goes through the registry, never directly to a file offset. That single indirection is what buys three separate requirements at once: SC-016 (corrections do not orphan references), FR-039 (a redacted PID resolves to a tombstone everywhere, including inside previously stored answers, because the render path re-resolves), and the loop-safety property in Finding 6.

Paired mutation proof (FR-032, SC-012). The SC-016 gate is: record N cross-references and citations → correct a transcript passage → re-index → assert all N resolve to the same PID and the same passage. Its mutation is: run the same gate against a build configured to key references on content_hash instead of pid, and require the gate to **FAIL**. Without that second half the gate proves nothing.

Alternatives considered

- **Content hash as identity.** Rejected — it is what Lumen does, and run 2 is the demonstration of why: a typo fix is a new identity. Explicitly forbidden by FR-037.
- **file:line as identity.** Rejected — run 3 is the demonstration: inserting a paragraph moved every later passage. Explicitly forbidden by FR-037.
- **Fuzzy re-matching on re-index (embedding similarity between old and new passages).** Rejected as the *primary* mechanism. It is probabilistic, so SC-016's "100 %" could never be honestly claimed,

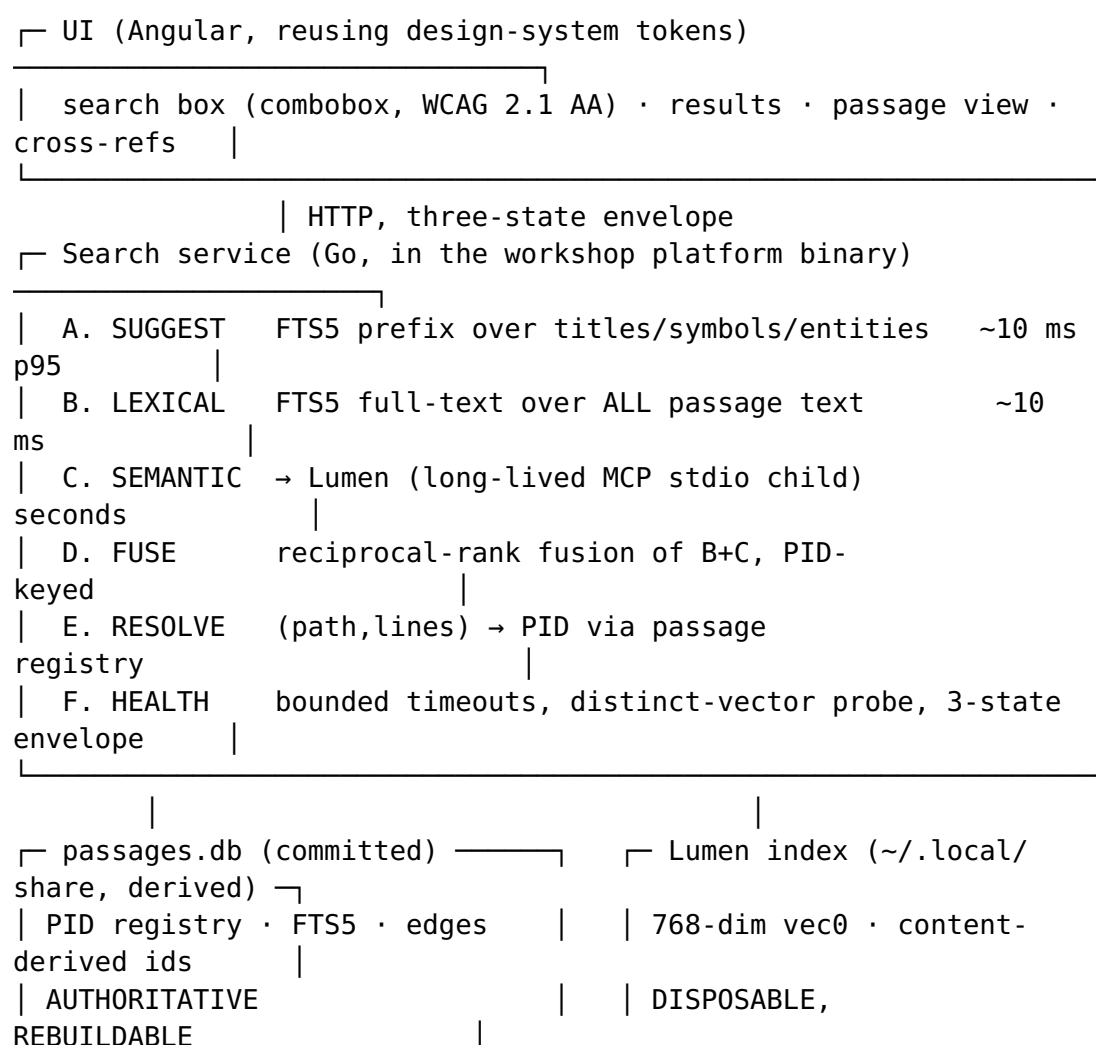
and its failures are exactly the silent kind the constitution forbids. Retained only as a *diagnostic*: when rule 4 mints a new PID for content that is ≥ 0.97 cosine-similar to a passage that just disappeared, the ingest run emits a warning for human review rather than guessing.

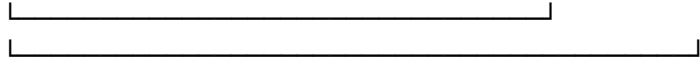
- **Sequential integer ids.** Rejected — stable, but only within one machine’s build order; two maintainers ingesting chapters independently would collide. ULID removes the coordination requirement.

Finding 3 — Reuse vs build: where the seam goes

Decision

Four layers. Lumen occupies exactly one of them.





Rationale

What must be built, and why each item cannot come from Lumen:

Capability	Why Lumen cannot supply it
Stable passage identity	Finding 2 — content-derived, and not exposed
Autocomplete ≤ 200 ms	Finding 5 — the cheapest path through Lumen is one CPU embedding
Coverage of .sh, .html, .css, PDFs, transcript timecodes	Finding 1 — not in the file-type allowlist
Cross-references	No such concept in the schema or the tool surface
Three-state availability	Finding 7 — <code>health_check</code> reported “healthy” during a hard embedding failure
An HTTP API a browser can call	Only a CLI and an MCP stdio server exist
Atomic index generations	Finding 8 — Lumen mutates its store in place while serving reads

What is genuinely reused: the embedding model and its 15 hours of already-computed vectors; the eight-language tree-sitter chunker; the merkle freshness walk; and the operational hardening already written in this repository — `scripts/lumen-reindex.sh` (GPU refusal, retry, and the only probe that catches the “repeated stale vector” corruption mode), `scripts/lumen-index-doctor.sh` (read-only distinctness audit, three-valued exit), `scripts/ollama-vulkan-remediation.sh`, `scripts/ollama-tune.sh`.

Transport to Lumen: a long-lived `lumen stdio` child process, not `lumen search per query`. This is a measured requirement, not a preference. The CLI’s `--trace` shows a fixed **2,202 ms “indexer setup”** phase per process:

```
[ 13ms] path resolution
[2202ms] indexer setup          → db opened, model ordis/jina-
embeddings-v2-base-code
```

```

[ 12ms] merkle + freshness      → index is fresh (no reindex)
[21040ms] query embedding      → 768 dims
[   3ms] knn search            → 1 candidates fetched (limit=3,
fetch=6)
[   0ms] post-processing
[23272ms] total

```

2.2 s of process warm-up alone consumes the whole SC-006 budget before a single vector is computed. Over the MCP stdio server, initialize costs **12–20 ms** and health_check **2 ms**, so the warm-up is not paid per call. (index_status is a different matter — see Finding 8.)

Line-range → PID resolution. Lumen returns (file_path, line-start, line-end). The service resolves that to a PID by an interval lookup in the registry, whose locator for document passages is the anchor-delimited line range recomputed at ingest. Overlap, not equality, is the match rule — Lumen’s chunk boundaries are its own (mean 5.8 lines) and need not coincide with passage boundaries. Where a Lumen chunk spans two passages, both PIDs are returned and de-duplicated by rank fusion. Where it resolves to none (a file type we index lexically but Lumen does not, or vice versa), the result is dropped with a counter incremented — a silently dropped result is a defect, and the counter is what makes it visible.

Alternatives considered

- **Read Lumen’s SQLite directly from the search service** (bypassing the CLI/MCP, doing our own vec0 k-NN). Tempting — knn search costs 2–3 ms — but rejected: Lumen mutates that database in place with BEGIN IMMEDIATE transactions and per-file DELETE FROM vec_chunks WHERE id IN (SELECT id FROM chunks WHERE file_path = ?), and its own binary carries a corrupted database detected during reindex, rebuilding path. Reading someone else’s live write-ahead-logged store, whose schema is undeclared and whose id semantics we have already shown to be unstable, trades 2 s of latency for an unbounded correctness risk. The service still needs its own embedding call for the query either way.
- **A parallel project-owned vector index for everything** (embed passages ourselves into passages.db via sqlite-vec). Rejected as the primary design because the spec commits to reuse, but explicitly **retained as the contingency**: it is the mitigation if the plugin is upgraded incompatibly, and the corpus size (a few thousand curriculum passages, versus 58,734 monorepo chunks) makes it cheap. The PID layer makes this switch a configuration change,

because nothing above layer C knows which engine produced a match.

Finding 4 — How each content type becomes passages

Decision

Every content type is normalised to **markdown with PID anchors, committed to the repository**, before indexing. Lumen then indexes the markdown. Nothing is fed to Lumen that Lumen cannot parse, and nothing is claimed to be indexed that is not.

Rationale, per type

Content	Passage unit	Semantic (Lumen)
Transcript segments	ASR segments merged into topic blocks of ~150–400 words, each a heading ### HH:MM:SS – Speaker: topic with a PID anchor and {t_start,t_end} in the registry	Yes — published as .md, indexed natively
Markdown documentation	Lumen’s own heading section, aligned to an authored PID anchor per section	Yes — native, already 92.7 % of the live index

Content	Passage unit	Semantic (Lumen)
---------	--------------	------------------

Source code	Lumen chunk: function / method / type / const / var / interface; PID keyed on symbol path	Only for .go .py .js .mjs .ts .tsx .jsx .json .yaml
-------------	---	--

Diagrams and schematics	The diagram's source text (mermaid / dot / plantuml) plus an authored caption and description, emitted as diagrams/ <name>.md with a PID anchor	Yes, via the generated .md
----------------------------	--	----------------------------

Content	Passage unit	Semantic (Lumen)
The notes PDF (Notes by Gemini.PDF, 417,800 bytes)	One passage per logical section, extracted with pdftotext (poppler-utils, available) into chapters/01/notes.md with PID anchors	Yes , via the generated .md
The recording (1.87 GB MP4, 36 × 50 MB parts)	Not a passage. It is a <i>target</i> : transcript passages carry {t_start,t_end} and deep-link to it (FR-002, SC-003)	No

Content	Passage unit	Semantic (Lumen)
---------	--------------	------------------

The normalisation rule, stated once

If Lumen cannot index it, generate a committed markdown derivative that Lumen can index — and give the derivative its own PIDs, provenance, and a link back to the original.

This is preferred over extending Lumen for three reasons: the derivative is human-reviewable in a diff; it survives a Lumen upgrade or replacement; and it forces the “what is genuinely indexable” question to be answered explicitly, in a file someone can read, rather than buried in an extractor’s behaviour.

Alternatives considered

- **Feed transcripts as .vtt/.srt.** Rejected — not in the allowlist (measured: 0 indexed), and the timing metadata is better held in the registry than in the indexed text, where it would pollute the embedding.
 - **Index raw video/audio with a multimodal embedding model.** Rejected — no such model is installed, the backend is CPU-only, and D1 forbids sending content off-machine. Recorded as out of scope rather than as a future promise.
 - **Skip the derivative and add a bespoke extractor per type feeding a private vector table.** Rejected for now — it is the Finding 3 contingency design, and adopting it early would abandon the reuse commitment for content types (diagrams, PDFs) that markdown handles adequately.
-

Finding 5 — Autocomplete at 200 ms p95 (SC-005)

Decision

Suggestions never touch the embedding backend. Type-ahead is served entirely from a SQLite FTS5 prefix index held by the search service process. The semantic path is not on the keystroke path at all.

The two paths return *different kinds of thing*, and that difference is the design, not a compromise:

- **Suggest (per keystroke, ≤ 200 ms)** returns **navigational targets and query completions**: chapter titles, transcript section headings, speaker names, code symbols, document titles, glossary terms, and previously-run queries. Each carries a PID (or a route) so Enter on a suggestion navigates *directly* to the passage without a search at all.
- **Search (on submit, ≤ 2 s)** returns **ranked passages by meaning**, fusing the lexical and semantic legs.

Rationale — both halves measured

The semantic path cannot meet 200 ms, and is not close. Measured on this machine, under concurrent index load:

```
$ for i in 1..5; do time curl -s -X POST localhost:11434/api/embed \
    -d '{"model":"jina-embeddings-code-cpu:latest","input":"how
do we archive the workshop recording"}'
18.28 s    15.41 s    0.10 s    0.16 s    0.24 s          ← same input
repeated; last three are cache-assisted
```

```
$ # ten DISTINCT short queries, same conditions
33.33 s ← "video archive pipeline design"
49.65 s ← "how are chapters organised in the curriculum"
28.75 s ← "constitution rule about server side CI"
(run aborted at the 2-minute cap)
```

lumen search --trace attributes it precisely: **query embedding 18,162 ms and 21,040 ms** in two runs, against 2–3 ms for the k-NN itself. Through the MCP server, one semantic_search returned in 9,563 ms and another failed at 20,013 ms.

The honest range is therefore: **~100–240 ms in the best observed case (idle-ish, cache-assisted, identical input), 9–50 s under the load this system creates for itself after every chapter ingest.** Even the best

case consumes the entire 200 ms budget with zero margin for HTTP, JSON and render. The true idle cold-query cost is **UNVERIFIED** (Appendix C-1) because measuring it requires stopping the running rebuild, which is prohibited — but no plausible value of it rescues the semantic path, because the *variance* is the disqualifier, not the mean. A p95 target cannot be met by a path whose p95 is set by an unrelated background job.

The lexical path meets it with two orders of magnitude to spare. A realistic suggestion dictionary was built from the 58,726 real symbols and headings in this repository's live Lumen index, copied read-only into the scratchpad:

```
source chunk rows: 58726
build time: 0.93s   db size: 15.9 MB
prefix-suggest latency over 400 probes (ms):
  min=0.052 p50=0.254 p95=9.580 p99=19.203 max=51.536
```

Probes were 2–5 character prefixes drawn from real symbol vocabulary — i.e. the worst case (a 2-character prefix matches thousands of rows) is included. **p95 = 9.6 ms**, measured while the machine was under load average ~9 *and* running the full re-index. That leaves ~190 ms of the SC-005 budget for HTTP, serialisation and paint.

Schema:

```
CREATE VIRTUAL TABLE suggest USING fts5(
  term, kind, pid UNINDEXED, route UNINDEXED,
  tokenize='unicode61', prefix='2 3 4'
);
-- query: SELECT term, kind, pid FROM suggest WHERE suggest
MATCH ?||'*' ORDER BY rank LIMIT 8;
```

No new dependency is required. FTS5 with prefix indexes was verified working in modernc.org/sqlite — the exact pure-Go driver already in ai_interviewing/platform/backend/go.mod:

```
$ go run .      # modernc.org/sqlite, CREATE VIRTUAL TABLE ... USING
fts5(..., prefix='2 3 4')
FTS5 OK, prefix match -> video archive pipeline
```

Client-side rules that the 200 ms figure depends on:

- Debounce **80 ms** (not the 220 ms used by ai_interviewing/platform/frontend/src/app/features/search.component.ts; at 10 ms server time the debounce would become the dominant term).
- Every in-flight suggest request is cancelled by the next keystroke (switchMap), and responses are dropped if a newer keystroke has

been issued — otherwise out-of-order responses produce a flickering list that reads as latency.

- **Minimum 1 character, not 2.** A single character is a legitimate prefix and the index handles it; the spec's edge case requires an empty query, a single character, and a multi-paragraph query all to behave predictably. Empty → recent queries plus chapter list. Single char → prefix matches. Multi-paragraph → suggestions suppressed (a paragraph is not a prefix), submit path only, and the input is truncated for the embedding at the model's 8192-token context.
- **Suggestions are rendered as an ARIA combobox:** `role="combobox"`, `aria-expanded`, `aria-controls`, `aria-activedescendant`, `popup` `role="listbox"`, Down opens/moves, Up/Down navigate, Enter selects, Esc closes — the pattern already written down in `design-toolkit/knowledge/aria-apg.md`. Focus never leaves the input, which is what satisfies FR-042 and the screen-reader edge case; the count of suggestions is announced in an `aria-live="polite"` region.
- **No such component exists in this repository today** (searches for `combobox`, `aria-activedescendant`, `typeahead`, `role="listbox"` return only prose). `design-system/components-extended.css` provides `.od-search`, `.od-search__input`, `.od-search__icon`, `.od-search__clear` — presentation only, no `listbox`. The combobox is new work built on those tokens.

Optional trailing semantic preview. After 350 ms of typing inactivity, the client *may* issue a cancellable semantic query and merge results *below* the lexical suggestions, clearly separated. It is strictly additive: it never blocks, never reorders the fast list, and its absence is invisible. SC-005 is measured on the lexical list only, and the measurement harness asserts that — otherwise a slow preview would silently pollute the percentile.

Alternatives considered

- **Cache query embeddings.** Rejected as the primary mechanism — it only helps repeated queries, and a prefix stream is by construction almost all novel strings. Retained as a small LRU for the submit path, where repeated queries are common.
- **A second, tiny embedding model for suggestions.** Rejected — a smaller model still costs a CPU forward pass and a backend round trip per keystroke, still contends with indexing, and adds a second model whose relevance is uncalibrated against the main index.
- **Precompute embeddings for a fixed suggestion vocabulary.** Rejected — the query still has to be embedded to compare against them. It solves the wrong half.

- **Trigram/LIKE '%q%', as ai_interviewing does today**
(store.go:501-509). Rejected — unanchored LIKE cannot use an index, has no ranking, and the existing implementation returns no snippets and no relevance order. FTS5 costs nothing extra and gives BM25 ranking.
-

Finding 6 — Cross-references (FR-018)

Decision

Cross-references are **stored directed edges between PIDs**, computed offline at ingest, in four kinds, with authored edges outranking derived ones and never being overwritten by recomputation.

```
CREATE TABLE passage_edge (  
  src_pid TEXT NOT NULL REFERENCES passage(pid),  
  dst_pid TEXT NOT NULL REFERENCES passage(pid),  
  kind TEXT NOT NULL, -- structural | explicit | similar |  
                        curated | contradicts  
  score REAL, -- similar: cosine; others NULL  
  origin TEXT NOT NULL, -- generated | authored  
  computed_at TEXT NOT NULL,  
  PRIMARY KEY (src_pid, dst_pid, kind)  
);
```

Rationale

Derivable automatically:

- **structural** — free and exact. Section parent/child, previous/next passage within a chapter, transcript passage ↔ the recording timecode it came from, transcript passage ↔ the overlapping region of the notes PDF, chapter ordinal ordering.
- **explicit** — free and exact. Markdown links, relative file references, and code symbol references already in the text, resolved to PIDs at ingest. A link whose target has no PID is recorded as a dangling edge with a reason, not dropped.
- **similar** — the semantic edges. Computed **at ingest, not at query time**: for each passage, k-NN against the index with $k = 12$, keep the top 5 above a cosine floor (start at 0.55, calibrate against the SC-007 benchmark), drop self and same-section neighbours. This is why it must be offline — at 9–50 s per embedding under load, computing neighbours on page load is impossible, and the passages are already embedded, so the query vector is free.

- **contradicts** — semi-automatic. Where a transcript passage and the notes PDF cover the same timespan with high similarity but a detected factual divergence, the pair is *flagged for review*; the edge is only written once a human confirms it. The spec requires disagreement to be surfaced, not resolved; a machine-asserted contradiction would be a resolution.

Must be authored:

- **curated** — the editorial links: “this exercise implements the pattern discussed in Chapter 1 §4”. These live in a committed `workshop/curriculum/crossrefs.yaml`, keyed by PID, and are merged in at build. Recomputation deletes and rewrites `origin='generated'` rows only; `origin='authored'` rows are never touched by a machine. Without that rule, one re-index would silently erase a maintainer’s editorial work.
- Diagram captions, and the “what this section is really about” summaries that make similarity edges useful rather than merely proximate.

Loop safety (spec edge case: “cross-references must not create loops that trap a reader”):

1. Edges are stored **directed** and rendered as a bounded list (max 5 per kind), never as an auto-followed chain.
2. The reader’s session keeps a visited-PID set; the immediately-preceding passage is suppressed from the offered list, so “related” never bounces the reader straight back.
3. Symmetric `similar` pairs are de-duplicated at render.
4. A build-time check reports strongly-connected components of size > 1 in the curated subgraph. It is a warning, not an error — a genuine mutual reference is legitimate; an unnoticed one is not.

Why PIDs are what make this work. Because an edge names two PIDs and never a file offset, correcting a transcript passage changes no edge; renaming a file changes no edge; re-indexing changes no edge. That is the mechanism by which SC-016 is satisfied for cross-references specifically, and it exists only because of Finding 2’s identifier layer.

Alternatives considered

- **Compute similarity at read time.** Rejected on the measured embedding latency, and unnecessary — the vectors already exist.
- **Store edges as markdown links inside the passage text.** Rejected — it makes every relationship a text edit, which mutates

content_hash, which triggers re-embedding, which makes editorial linking expensive. The edge table keeps relationships out of the content.

- **A graph database.** Rejected — the corpus is “one chapter, growing to tens” per the spec’s own scale assumption. A table with a composite primary key is the right size, and it is already in the file everything else lives in.

Finding 7 — Honest degradation: the three-state contract (FR-019, FR-020)

Decision

One type, in one place, with **no representation for “empty and healthy-unknown”**:

```
type SearchOutcome struct {
    Status    string           // "ok" | "no_match" |
                    "unavailable" (exhaustive)
    Results   []Hit           // non-empty iff Status == "ok"
    Degraded  *DegradedNotice // present when some legs answered
                    and others did not
    Reason    *Unavailable      // present iff Status ==
                    "unavailable"
}
```

Enforced in the search service, at the single function that constructs it. The HTTP layer maps unavailable → **HTTP 503** with a machine-readable reason, never 200 []. The client has three render branches and a test that fails if the no-results copy renders on a 503.

State	Condition	What the user sees
ok	≥ 1 result at or above the relevance floor	Ranked results. If a leg failed, a persistent banner: “Meaning-based search is unavailable right now — showing keyword matches only.”
no_match	All legs answered successfully; zero	“No passage in the curriculum matches that.” Plus the

State	Condition	What the user sees
	results above the floor	nearest below-floor items, explicitly labelled as below the relevance threshold — shown, but never presented as answers (FR-019).
unavailable	Embedding backend unreachable, saturated, timed out, returning degenerate vectors; or the index has no verified generation	“Search cannot answer right now” + reason + retry guidance. Never the empty-results copy.

Rationale — this repository has already produced the failure, twice, on both sides of the boundary

The good news: Lumen’s MCP surface distinguishes error from empty. Observed live during this research:

```
== semantic_search 20013 ms  q='quantum chromodynamics lattice
gauge theory' ==
embed query: all embedding servers exhausted after failover:
last error: embedding batch starting at 0: ollama embed: context
deadline exceeded [isError=True]
```

`isError=True`. A saturated backend is *not* silently rendered as zero results by Lumen itself.

The bad news, and it is the actual hazard: Lumen also emits a degradation warning glued to an empty-results string in the same unstructured blob. Observed:

```
call: 4991 ms -> "No results found. | Warning: Index is being
updated in the background.
                Results may be incomplete or outdated. Us..."
```

A client that reads the first line — or that renders the text as-is — shows “No results found.” The honest signal is present but not separable without string parsing. **The service must parse for that warning and promote it to Degraded, and must never forward Lumen’s text to the UI verbatim.**

Worse: health_check is a liveness probe, not a saturation probe, and it lies about saturation. Measured in the same MCP session, seconds apart:

== health_check 2 ms ==	== semantic_search 20013 ms ==
Status: OK	embed query: all embedding
servers exhausted...	
Message: service is healthy	[isError=True]

health_check reported “healthy” while queries were failing. It cannot be the source of truth for the third state. What must be used instead:

1. **A bounded timeout on the actual query.** `search_timeout_ms` (default 5,000; suggest path never calls it). Exceeded → `unavailable{reason:"embedding_timeout"}`. This is the direct fix for the recorded stall.
2. **The distinct-vector probe from `scripts/lumen-reindex.sh`** — send N distinct texts, require N distinct vectors back — run on a background timer (not on the request path) and cached. It is the only check in this repository that catches the corruption mode where the backend returns HTTP 200 with a *repeated stale* vector: well-formed, non-NaN, non-zero, correct L2 norm, and completely wrong. That mode put 758 duplicate vectors into this index on 2026-08-26.
3. **`index_status`’s Stale: field**, polled in the background (it costs 2.6–5.1 s on this repo — see Finding 8; it must never be on the request path).
4. **Indexer-running detection** → `Degraded{semantic:"reindexing"}`, results still served from the lexical leg.

The exact anti-pattern is already shipped in this repository and must not be copied. `ai_interviewing/platform/frontend/src/app/features/search.component.ts` handles a backend error by setting `loading(false)`, so a 500 renders as the empty state; and its backend returns `[]` for queries shorter than two characters. Both are the FR-020 violation. The workshop platform must not inherit them, and the plan should note that the `ai_interviewing` search page has the same defect independently of this feature.

Load isolation — an operational requirement, not a nicety. `scripts/ollama-tune.sh` records that with `OLLAMA_NUM_PARALLEL` resolving to 1, ollama serialises clients and “*a single embed went from 0.74s to a >90s client timeout, which stalled indexing entirely*”, with the design note “**queue depth is not the defect — serialisation is.**” This research reproduced exactly that: 28–50 s per query embedding while the

indexer held the backend. Therefore: interactive query embedding must have a **reserved parallel slot or a separate backend instance** from the indexing workload. Without it, SC-006 fails every time a chapter is ingested — which is precisely when people will be using the system.

Correction to the spec, offered honestly. The spec (line 145) and `plan.md` cite “*a saturated embedding backend caused queries to time out at 75 s.*” An exhaustive search of `docs/`, `_analysis/`, `CONTINUATION.md`, `.ashlrcode/`, `specs/`, `_tests/evidence/` and `.lumen-reindex.log` found **no artifact recording a 75 s figure**. The two saturation stalls that *are* recorded measure **>90 s** (`scripts/ollama-tune.sh:8-11`, embed serialisation) and **~10 minutes** (`docs/setup-agents-wizard/ARCHITECTURE.md:286-289`, `MANUAL.md:130-132`, `ACTION-REQUIRED.md:215-216` — an unbounded lumen search behind a running index stalled the setup wizard). The spec’s *conclusion* is right and this research independently confirms the failure class; only the number is untraceable. It should be corrected to cite the recorded measurements, or restated as “tens of seconds to minutes”.

Alternatives considered

- **Use `health_check` as the availability gate.** Rejected on the measurement above — it returned OK during a hard failure.
 - **Return 200 with `{"results": [], "degraded": true}`.** Rejected — every HTTP client, cache and proxy treats 200 as success, and the failure mode this requirement exists to prevent is precisely a successful-looking empty response. 503 makes the failure structural.
 - **Retry transparently until success.** Rejected — that is how a 10-minute stall happens. Retries are bounded (one, with jitter) and the remaining budget is reported as unavailable.
 - **Fail the whole query when the semantic leg is down.** Rejected — FR-025 requires browsing and search to continue when answering is unavailable, and the lexical leg is genuinely useful. Hence ok + Degraded rather than unavailable, with the degradation always visible.
-

Finding 8 — Index freshness, atomicity, and incremental extension (FR-026, FR-027)

Decision

Generations, verified before serving, swapped atomically. The serving index is never mutated in place.

A `index_manifest.json` names the current generation: {generation, built_at, source_commit, passage_count, edge_count, model, dims, checksum, verified}. The service opens the generation the manifest names; a build writes `search.<gen+1>.db`, verifies it, then rewrites the manifest by atomic rename. Readers re-open on generation change. A failed verification leaves the previous generation serving — the spec’s own rule, “a stale-but-consistent index is preferable to an inconsistent one”.

Verification gate, all four required: row counts match the manifest; every PID in the manifest resolves; the distinct-vector probe passes; and a smoke set of benchmark queries returns their expected PIDs. Only then is verified: `true` written.

Rationale — Lumen does not do this, and the difference must be absorbed, not assumed away

Measured during the running rebuild: while `lumen index -f` had been executing for 1 h 20 m, the live index **remained readable and was being mutated in place**:

```
$ sqlite3 "file:$D?mode=ro" "select count(*) from chunks;"      →
58734, then 58744
$ sqlite3 "file:$D?mode=ro" "select ... from project_meta;"
  last_indexed_at=2026-08-31T17:51:01Z      ← still the PREVIOUS
run's timestamp
  root_hash=11e7034b26c9b83c...             ← still the PREVIOUS
run's hash
$ index_status → "Last indexed: 2026-08-31T17:51:01Z | Stale: yes"
```

So during a rebuild the store holds a mixture of old and new rows while its freshness metadata still advertises the *previous* generation. That is a half-written, readable index — exactly the condition the spec forbids serving from. Lumen’s only mitigation is the soft text warning quoted in Finding 7. **Therefore the workshop platform’s generation discipline applies to its own artifacts (`passages.db`, the FTS5**

indexes, the edge table), and Lumen’s contribution during a rebuild is treated as `Degraded{semantic:"reindexing"}` — surfaced to the user, never silently trusted.

`index_status` is a background poll, not a request-path call. Measured on this repo through a warm MCP session, three consecutive calls:

```
index_status(vasic) call#1: 5147 ms
index_status(vasic) call#2: 2619 ms
index_status(vasic) call#3: 3310 ms
```

2.6–5.1 s, not amortised across calls. On the request path it would fail SC-006 on its own.

The extension procedure (FR-026, FR-027, FR-028, FR-029)

`workshop/scripts/ingest-chapter.sh <NN>` — one command, seven phases, each with a receipt:

Phase	Idempotency key	On re-run
1. Preflight	—	Re-checks materials; reports precisely what is missing and exits non-zero without publishing (FR-028)
2. Reassemble + verify recording	recording sha256 vs .sha256 manifest	Skips; <code>workshop/scripts/extract-videos.sh</code> already verifies at three levels
3. Transcribe	recording sha256	Skips. Segment-level checkpoints make it resumable (FR-029) — see <code>research/transcription.md</code>
4. Mint / reconcile PIDs	presence of <code><!-- pid: ... --></code> anchors	No-op — anchors already exist, so nothing is minted. This is what makes FR-027 true rather than merely tested.
5. Publish curriculum pages	content hash of inputs	

Phase	Idempotency key	On re-run
6. Index delta	file sha256 (Lumen's own files.hash)	Regenerates; byte-identical output $\Rightarrow$ no commit Lumen re-embeds only changed files — measured: run 2 of the Finding 2 experiment reported Files: 0 added, 1 modified, 0 removed (2 total), Indexed 1 files, 3 chunks, leaving b.md untouched. A new chapter costs only its own files.
7. Cross-refs + verify + evidence	generation number	New passages are k-NN'd against the existing index; existing$\rightarrow$new edges added by symmetry. The existing$\times$existing set is not recomputed. origin='authored' edges are never touched.

Each phase writes a .receipt containing the digest of its inputs; a re-run whose digest matches is a no-op that prints why it skipped. This is the same pattern workshop/scripts/archive-videos.sh already uses (already_archived() re-hashes the video and every part), and workshop/scripts/self-test.sh already asserts idempotency for archive and extract — the ingest self-test extends it to the whole chain, including a mutation that corrupts one input and requires the run to fail.

Redaction propagation (FR-039). A redaction sets redacted_at on the PID. The next generation excludes the passage from all indexes, and — because every citation and cross-reference resolves *through* the registry at render time — previously stored answers immediately

render the citation as a tombstone rather than the text. A redaction that only edited the transcript would leave the vectors and any cached answers intact; PID indirection is what prevents that.

Alternatives considered

- **Serve Lumen's index directly and rely on its "Stale" flag.** Rejected on the measurement above: Stale: yes was reported for 80 minutes while rows changed underneath, and the flag costs 2.6–5.1 s to read.
 - **Lock the index during rebuild and return unavailable.** Rejected — a full monorepo rebuild takes hours on this hardware; hours of total unavailability is worse than degraded-but-honest service from the previous generation plus the lexical leg.
 - **Full rebuild on every chapter.** Rejected — measured incremental behaviour already scopes work to changed files, and a full rebuild is the ~15-hour operation the `.lumenignore` file was written to reduce.
-

Consolidated recommendation

1. **Keep Lumen. Add a PID layer above it.** Lumen is a good embedding-and-k-NN engine for markdown and eight code languages. It is not a search system, and its chunk ids fail FR-037 by measurement, not by opinion.
2. **PIDs are ULIDs minted at ingest and written into the source artifacts as anchors**, with a committed registry (`passages.jsonl` → `passages.db`). Every citation, cross-reference and redaction resolves through the registry. This is what makes SC-016 mechanically true.
3. **Two retrieval paths with different budgets:** FTS5 prefix suggestions (**measured p95 9.6 ms**) on every keystroke; fused lexical + semantic on submit. The embedding backend never sees a keystroke.
4. **Normalise every content type to committed markdown** before indexing, and document the real gaps: `.sh/.html/.css` are lexical-only; raster diagrams give OCR labels but not structure; the recording is a link target, not a searchable surface.
5. **Three states, one type, 503 for unavailable.** Do not trust `health_check` — it reported healthy during a hard failure. Use bounded timeouts plus the distinct-vector probe.
6. **Reserve embedding capacity for interactive queries** separately from indexing, or SC-006 fails every time a chapter is ingested.

7. **Generations with a verification gate and atomic swap** for everything this feature owns; treat a Lumen rebuild as a declared degraded state.

Consequences for the success criteria

Criterion	Assessment on this evidence
SC-005 (suggestions $\leq$ 200 ms p95)	Achievable with margin — 9.6 ms p95 measured on a 58,726-row prefix index under load. Only achievable because the path is lexical.
SC-006 (results $\leq$ 2 s p95)	Achievable only under conditions that must be engineered: a long-lived MCP child (avoids the measured 2.2 s per-process warm-up) <i>and</i> embedding capacity reserved from indexing. Not achievable while a full re-index holds the backend — where 9–50 s was measured. The degraded state exists for exactly that window.
SC-007 ($\geq$ 90 % top-5)	Not measurable before the corpus exists. The benchmark must be versioned (<code>benchmark/queries.yaml</code> → <code>workshop/platform/backend/testdata/benchmark/retrieval.tsv</code> , path and format corrected 2026-09-01 — see tasks.md File Structure; $\geq$ 20 queries, each with an expected PID) and run as a gate with evidence written to <code>_tests/evidence/</code> .
SC-008 ($\geq$ 80 % with no literal overlap)	Same, plus: the harness must <i>machine-verify</i> zero token overlap between each flagged query and its target passage text, so “shares no literal words” is proved rather than asserted.

Criterion	Assessment on this evidence
SC-016 (100 % resolve after correction + re-index)	Achievable, and only via the PID layer. With Lumen ids it would be 0 % for every corrected passage. Gate + paired mutation described in Finding 2.

Appendix A — Commands and raw output

```
# Version, CLI surface
lumen version                                # →
0.0.41

lumen --help ; lumen index --help ; lumen search --help

# Locate this repo's index among 765 stores (2.6 GB total)
for d in ~/.local/share/lumen/*/; do
    sqlite3 "file:${d}index.db?mode=ro" \
        "select value from project_meta where key='project_path';"
done
# → <ext-volume-host>/Projects/vasic = 21bf1507a8925bcf

D=~/.local/share/lumen/21bf1507a8925bcf/index.db
sqlite3 "file:$D?mode=ro" ".schema"          #
    schema in Finding 1
sqlite3 "file:$D?mode=ro" "select count(*) from chunks;" # →
58734 ... 58744 (live rebuild)
sqlite3 "file:$D?mode=ro" "select count(*) from files;" # →
2510
sqlite3 "file:$D?mode=ro" "select kind,count(*) from chunks group
by kind order by 2 desc;"
sqlite3 "file:$D?mode=ro" "select length(id),count(*) from chunks
group by 1;" # → 16|58734
sqlite3 "file:$D?mode=ro" \
    "select 'min='||min(end_line-start_line+1)||' avg='||
    round(avg(end_line-start_line+1),1)
    ||' max='||max(end_line-start_line+1) from
    chunks;" # → min=1 avg=5.8 max=133

# File-type coverage (0 for every one of these)
for ext in .sh .html .css .txt .svg .mmd .puml .vtt .srt .pdf .mp4 .rb .toml .ipynb; do
    sqlite3 "file:$D?mode=ro"
        "select count(*) from files where path like '%$ext';"
done
git ls-files '*.sh' | wc -l # → 55
git ls-files '*.html' | wc -l # → 146
git ls-files '*.css' | wc -l # → 32

# files.hash is sha256 of file bytes
```

```
sqlite3 "file:$D?mode=ro" "select hash from files where
    path='workshop/README.md';"
sha256sum workshop/README.md      # identical → 3f83a338daa79e39...
```

Backend state

```
ollama ps      # jina-embeddings-code-cpu:latest  345 MB  100% CPU
                CONTEXT 8192
ollama list    # jina-embeddings-code-cpu:latest ; ordis/jina-
                embeddings-v2-base-code:latest
ps -eo pid,etime,args | grep lumen
#   897646  01:20:56  lumen-linux-amd64 index -f /run/media/.../
                vasic      ← running throughout
```

Embedding latency, under concurrent index load:

```
for i in 1..5; do /usr/bin/time -f "%e" curl -s -o /dev/null -X
    POST \
    http://localhost:11434/api/embed -H 'Content-Type: application/
        json' \
    -d '{"model":"jina-embeddings-code-cpu:latest","input":"how do
        we archive the workshop recording"}'; done
# 18.28  15.41  0.10  0.16  0.24      (identical input; last three
    cache-assisted)
```

```
# ten DISTINCT queries – aborted at the 120 s cap after three
# 33.33 s · 49.65 s · 28.75 s
```

lumen search --trace (own scratchpad project, 2 files):

```
[ 13ms] path resolution
[2202ms] indexer setup          → db opened, model ordis/jina-
embeddings-v2-base-code
[ 12ms] merkle + freshness      → index is fresh (no reindex)
[21040ms] query embedding       → 768 dims
[   3ms] knn search             → 1 candidates fetched (limit=3,
fetch=6)
[   0ms] post-processing        → merged 1→1 results, filled 1
snippets
[23272ms] total
```

MCP stdio session (driven directly, ids matched):

```
initialize                20 ms
health_check              2 ms  → "Status: OK / service
is healthy"
index_status (2-file project) 2585 ms  → "Files: 2 | Indexed: 2
| Chunks: 5 | Stale: no"
index_status (vasic) x3      5147 / 2619 / 3310 ms → "Stale:
yes"
semantic_search (hit)        9563 ms  → Found 1 results ...
[isError=None]
semantic_search (saturated) 20013 ms  → "embed query: all
embedding servers exhausted after
```

```
failover: ... context
deadline exceeded" [isError=True]
semantic_search (reindexing) 4991 ms → "No results found. |
Warning: Index is being updated
in the background.
Results may be incomplete or outdated."
```

Identifier-stability experiment — full script at `scratchpad/idstab-run.sh`; transcript in Finding 2. Fixture was a fresh `git init` project of two markdown files in the scratchpad; the live index was never written to.

FTS5 prefix suggestion benchmark (58,726 real symbols copied read-only out of the live index):

```
build time: 0.93 s      db size: 15.9 MB
400 probes, 2–5 char prefixes drawn from real symbol vocabulary:
min 0.052 · p50 0.254 · p95 9.580 · p99 19.203 · max 51.536 (ms)
```

FTS5 availability in the driver already in `go.mod`:

```
$ go run . # modernc.org/sqlite; CREATE VIRTUAL TABLE ... USING
fts5(..., prefix='2 3 4')
FTS5 OK, prefix match -> video archive pipeline
```

Appendix B — Existing repository assets this design builds on

Asset	Role in this design
<code>scripts/lumen-reindex.sh</code>	The index build wrapper. Its distinct-vector probe (N texts $\rightarrow N$ distinct vectors) is adopted as the health probe in Finding 7 — it is the only check that catches the repeated-stale-vector corruption mode. Note it has no lockfile ; concurrency safety is delegated to Lumen.
<code>scripts/lumen-index-doctor.sh</code>	Read-only corruption audit with a three-valued exit (0 healthy / 1 corruption / 2 could not inspect). The same three-valued convention is what FR-033 requires and is adopted for every gate here.
<code>scripts/ollama-tune.sh</code>	Records the serialisation defect (>90 s embed under <code>OLLAMA_NUM_PARALLEL=1</code>) that motivates the reserved-capacity requirement.

Asset	Role in this design
scripts/ollama-vulkan-remediation.sh	Why the backend is CPU-only, and therefore why embedding costs what it costs.
.lumenignore	Precedent for scoping the index; currently excludes <code>_tests/evidence/</code> . New generated derivatives must be reviewed against it.
ai_interviewing/platform/backend (Go 1.26, gin, modernc.org/sqlite, goldmark)	The structural template (FR-013) and the source of the verified FTS5 capability. Its <code>scripts/{start,stop,restart,status,build,ingest}.sh</code> are the shape the workshop platform's scripts should take. It is not containerised — no Dockerfile or compose file exists in it.
ai_interviewing/.../features/search.component.ts + store.go:501-509	The anti-pattern : LIKE '%q%', no ranking, no suggestions, and an error handler that renders failures as the empty state. Documented here so it is not copied.
design-system/components-extended.css	<code>.od-search*</code> — input, icon, clear button, focus ring. Presentation only; no listbox.
design-toolkit/knowledge/aria-apg.md	The written combobox keyboard/ARIA contract to implement against for FR-041/FR-042.
workshop/scripts/{archive,extract}-videos.sh, self-test.sh	The checksum-verified reassembly (D3) and the idempotency/corruption test pattern reused in Finding 8.

Appendix C — UNVERIFIED register

Each item states what could not be established and why, per FR-033 and the prohibition on reporting unverified states.

- **C-1 — Idle (uncontended) query-embedding latency.** All measurements were taken while `lumen index -f` held the backend. The only sub-second figures (0.10–0.24 s) came from repeating an *identical* input, which is cache-assisted and therefore not representative of a novel query. **Reason not measured:** obtaining it requires stopping the running rebuild, which the research

constraints prohibit. **Impact:** the SC-006 headroom estimate is a lower bound on cost, not a measurement. **How to close:** after the rebuild completes, run 20 distinct queries against an idle backend and record the p95.

- **C-2 — Lumen’s chunk-id derivation formula.** 120 candidate derivations were tested against four known pairs; none matched. **What this does not affect:** Finding 2’s conclusion, which is behavioural (observed across three re-index runs) and does not depend on knowing the formula. **What it does affect:** any hypothetical scheme that would try to *predict* Lumen ids. Such a scheme is rejected anyway.
- **C-3 — Whether the MCP `semantic_search` path amortises the 2.2 s “indexer setup” across calls in one session.** `initialize` costs 12–20 ms and `health_check` 2 ms, so it is not paid at startup; but `index_status` cost 2.6–5.1 s on *each* of three consecutive calls in one warm session, which is inconsistent with full amortisation. The binary contains an `indexer cache hit` string, suggesting caching exists under some conditions. **Impact:** the SC-006 budget may or may not carry a multi-second fixed cost per query. **How to close:** after the rebuild, time ten consecutive `semantic_search` calls in one MCP session against an idle backend and inspect the spread.
- **C-4 — Merkle freshness-walk cost on the full 2,510-file repo.** Measured at 10–12 ms on a 2-file project only; the corresponding figure for the monorepo was not isolated (it is folded into the 2.6–5.1 s `index_status` numbers). **Reason not measured:** isolating it requires running `lumen search` against the vasic index during the rebuild, which risks triggering index work on a store that must stay read-only.
- **C-5 — SC-007 and SC-008 retrieval quality.** Not measurable: the transcript does not exist yet, so there is no corpus and no benchmark. Nothing in this document claims a quality figure. The `min_score` floor (Lumen’s default is model-dependent; the binary carries `computeMaxDistance` and `boostedScore` logic that is not documented) must be **calibrated against the benchmark once it exists**, not guessed. Observed scores on a two-document toy index were 0.69 for a strong match and 0.41 for a weak one, which is far too small a sample to set a threshold from.

- **C-6 — Frame-OCR usefulness for on-screen text in the recording.** `tesseract-ocr` and `ffmpeg` are available, so the pipeline is constructible, but its precision and false-positive rate on this recording are unmeasured. It is recorded as feasible-and-optional, not as planned capability.
- **C-7 — The spec’s “75 s” saturation figure.** No artifact recording it was found anywhere in the repository. The recorded measurements are >90 s and ~10 min. See the correction note in Finding 7. The failure *class* is verified; the specific number is not.